Assignment 1 Smart Library Architecture Prototype
CSCS5103 Software Architecture | Individual work | 5 points

Purpose.
Build a small working library prototype and show how a layered structure helps keep user interaction, business rules, and data storage separate. By the end, you should be able to trace a borrow request through the layers and explain why that separation makes later changes easier.

Due date and submission.
Submit one Word document or one PDF through the Assignment 1 submission area in MS Teams by the date shown there. Submit your own work. Do not submit a ZIP archive, separate code files, or a link that requires the marker to download other materials.

1 Scenario
A campus library wants a simple prototype for its basic circulation desk. A librarian should be able to register a reader, view registered readers, add a book, and process a borrowing request. The prototype is successful when a reader can be registered, a book can be added, and a valid borrowing request changes that book's availability in the running application.
Keep the scope deliberately small: use in-memory storage, so records may reset when the application restarts. Do not add authentication, a database, a web interface, deployment, microservices, or extra features.

2 Architecture and scope
Design a layered application with a presentation layer, a business layer, and a data-access layer. Within the business layer, use components such as UserService and BookService to hold the application rules. These are ordinary components in one application; this assignment does not require independently deployed services or a microservices architecture.
Your diagram should make the request path and permitted dependencies clear. For example, a controller handles an HTTP request, calls a service, and the service uses a repository or in-memory store. The presentation layer must not read or update the store directly.

3 Required operations
Implement four REST endpoints so the complete library workflow can be demonstrated. Use resource-oriented paths and appropriate HTTP methods. The paths below are examples; equivalent, clearly named routes are acceptable.

Operation	Example route	Expected result
Register a reader	POST /users	Creates a reader and returns an identifier or confirmation.
List readers	GET /users	Returns the readers currently stored in memory.
Add a book	POST /books	Creates a book with an identifier and available status.
Borrow a book	POST /loans	Checks the reader and book, records the loan, and marks the book unavailable.

A borrow request should be rejected with a clear response if the reader or book does not exist, or if the book is already unavailable. No additional endpoints are required.

4 Work to complete
- *Architecture diagram and explanation*. Draw the layers and name the main components, such as UserController, BookController, UserService, BookService, and repositories or in-memory

stores. Add arrows to show the request flow and dependency direction. Write 100–150 words explaining the responsibilities, one important design decision, and how the request moves through the application.

- *Implementation*. Implement the four operations using Java, Python, or another language and framework suitable for a small REST prototype. Use in-memory storage. Keep controllers, business logic, and data access in distinct components. Include complete source code in your submitted document, with each source filename clearly labelled. The code listing must be readable and complete enough to follow the implementation.

- *Run and test instructions*. Include the equivalent of a short README: prerequisites, the command to start the application, and how to call each endpoint. Then show one end-to-end test using example request and response data: register a reader, add a book, borrow that book, and show its updated unavailable status. Include one unsuccessful borrow case, such as an unknown reader or an already-borrowed book, with the response. Text transcripts are acceptable; screenshots may supplement them.

- *Quality table and reflection*. Rate the architecture for modifiability, scalability, reusability, and maintainability using a consistent scale (for example, 1–5). Add a brief reason for each rating. Write about 100 words on how the design supports modularity and one change you would make if the system later needed to support substantially more users or data. Keep the discussion tied to the prototype and its trade-offs.

5 What to include in your single file

1. Your name and student ID on the first page.
2. The architecture diagram and 100-150-word explanation.
3. All implementation source code, grouped under the source filename for each listing.
4. Run instructions and the request/response evidence for the required tests.
5. The four-row quality table and approximately 100-word reflection.
6. A short AI-use note if you used permitted AI support, stating the tool and purpose.

Place the diagram, text, code, and test evidence directly in the same Word document or PDF. Use a monospaced font for code. Check that the exported PDF preserves code, diagram labels, and test output before you submit it.

6 Assessment rubric

Total: 5 points. The rubric below is used to assess the submitted document and the evidence it contains.

Criterion	Points	Evidence for full credit
Architecture design	1.0	Diagram clearly shows the three layers, named components, request path, and allowed dependencies (0.5). The 100–150-word explanation assigns responsibilities and justifies a design decision (0.5).
Implementation	3.0	The four required operations form a working end-to-end flow and handle the specified borrow checks (1.5). Controllers, services, and data access are separated and responsibilities are placed appropriately (0.75). Code is coherent, uses in-memory storage, and returns understandable outcomes (0.5). Run instructions and test evidence are complete enough to verify the workflow from the document (0.25).
Quality and reflection	1.0	The table rates all four required characteristics and gives a plausible reason for each rating (0.4). The reflection explains modularity and one realistic scaling change, with a sensible connection to trade-offs (0.6).

Academic integrity and AI use

This is an individual assignment. You may use generative AI for brainstorming, explaining course concepts, or improving language. You may not use it to create the assignment solution, including the architecture diagram, design explanation, implementation code, test results, or reflection. State any permitted AI use briefly in your submission. Do not copy another student's work or submit work produced by another person.